Algorithm Cheat Sheet — All Templates + Java Syntax

Conventions, Skeleton & Core API

Generic Test Harness

```
class Question {
    private int data;
    public Question(int data) {
        this.data = data;
    }
    public void solve() {
        // TODO
    }
}
public class Solution {
    public static void
main(String[] args) {
        Question q = new
Question(0);
        q.solve();
    }
}
```

Naming Conventions (enforced across all template files)

Common Java API Cheat-Sheet

```
Map<Integer, Integer> map   =
new HashMap<>();      // key →
value
Map<Integer, List<Integer>> adj
= new HashMap<>();  //
adjacency / buckets
Set<Integer> set            =
new HashSet<>();        //
membership
List<Integer> list          =
new ArrayList<>();      //
dynamic array
Deque<Integer> stack = new
ArrayDeque<>();            //
STACK: push / pop / peek
Deque<Integer> queue = new
ArrayDeque<>();            //
QUEUE: offer / poll / peek

// (ArrayDeque beats
java.util.Stack / LinkedList)
PriorityQueue<Integer> minHeap
= new PriorityQueue<>();
// min-heap (default)
PriorityQueue<Integer> maxHeap
= new PriorityQueue<>
(Collections.reverseOrder());
// max-heap
PriorityQueue<int[]>   pq
= new PriorityQueue<>((a, b) ->
a[1] - b[1]);  // by field,
min-first
TreeMap<Integer, Integer> tmap
= new TreeMap<>();     // sorted
keys: floor/ceiling/first/last
TreeSet<Integer> tset
= new TreeSet<>();     // sorted
set: floor/ceiling/higher/lower
LinkedHashSet<Integer> lset
= new LinkedHashSet<>(); //
insertion-ordered set (LFU
buckets)
Random rand
= new Random();         //
rand.nextInt(n) → 0..n-1
(RandomizedSet)
int[]      arr  = new int[n];
// defaults to 0
boolean[]  seen = new
boolean[n];
// defaults to false
int[][]    grid = new int[m]
[n];                    // 2D,
all 0
int[]      dr   = {1,-1,0,0},
dc = {0,0,1,-1};  //
DOWN,UP,RIGHT,LEFT; iterate
for(dir=0..3)
i+dr[dir],j+dc[dir]
```

```
// —— Map ——
map.put(k, v);
map.getOrDefault(k, 0);
// safe read with default
map.merge(k, 1, Integer::sum);
// ← counting frequency (most
common)
map.computeIfAbsent(k, x -> new
ArrayList<>()).add(v); // ←
build adjacency / buckets
map.putIfAbsent(k, v);
// ← write only first
occurrence (keep earliest
index)
map.containsKey(k);
map.remove(k);
for (Map.Entry<Integer,Integer>
e : map.entrySet()) {
e.getKey(); e.getValue();
e.setValue(v); }
map.keySet();  map.values();
// —— Set ——
set.add(x);  set.contains(x);
set.remove(x);
// —— List ——
list.add(x);  list.get(i);
list.set(i, x);
list.remove(list.size() - 1);
// ← pop in backtracking
list.size();  list.isEmpty();
list.addAll(other);
Collections.sort(list);
// ascending
list.sort((a, b) -> b - a);
// custom / descending
Collections.reverse(list);
// ← reverse path after
building it backwards
```

```
// Deque as STACK (LIFO)
// Deque as QUEUE (FIFO)
stack.push(x);
queue.offer(x);
int top = stack.pop();
int head = queue.poll();
int look = stack.peek();
int look = queue.peek();
stack.isEmpty();
queue.isEmpty();
// Deque BOTH ENDS — needed for
the MONOTONE DEQUE (sliding-
window max/min)
// and for addFirst-style level
building (zigzag BFS).
deque.offerLast(x);
deque.pollLast();
deque.peekLast();    // back
(push/pop/peek tail)
deque.offerFirst(x);
deque.pollFirst();
deque.peekFirst();   // front
(push/pop/peek head)
deque.addLast(x);
deque.addFirst(x);
// same as offer*, throw if
capacity-bound
// PriorityQueue (heap)
pq.offer(x);  int best =
pq.poll();  int look =
pq.peek();  pq.size();
pq.remove(x); // ← O(n) removal
of an arbitrary element (lazy-
deletion alternative in #480)
```

```
Integer.MAX_VALUE;   //
2147483647  (2^31 - 1)   —
sentinel for "min so far"
Integer.MIN_VALUE;   //
-2147483648  (-2^31)       —
sentinel for "max so far"
Long.MAX_VALUE;      // when
sums can overflow int,
accumulate in long
int k = i + (j - i) / 2;
// ← avoid (i + j) overflow in
binary search
Integer.compare(a, b);
// ← overflow-safe comparator;
use instead of (a - b)
(a, b) -> Integer.compare(a[1],
b[1]);// ← safe
PriorityQueue/Arrays.sort
comparator
Double.compare(a, b);
// ← comparator for double[]
heaps (probability, median)
```

```
ch - 'a';
// 'a'..'z'  → 0..25
(lowercase bucket index)
ch - '0';
// '0'..'9'  → 0..9    (digit
value)
(char) ('a' + i);
// 0..25 → 'a'..'z'
num = num * 10 + (ch - '0');
// ← build a number digit by
digit
Integer.parseInt("123");
// String → int (throws on bad
input)
Integer.valueOf(123);
// int → Integer (boxed)
String.valueOf(123);
// int/char/etc → String
Character.getNumericValue('7');
// char → 7
// char classification
Character.isDigit(ch);
Character.isLetter(ch);
Character.isLetterOrDigit(ch);
Character.isWhitespace(ch);
Character.toLowerCase(ch);
Character.toUpperCase(ch);
```

```
s.length();  s.charAt(i);
s.substring(i, j);        //
[i, j)
s.toCharArray();  s.split(" ");
s.equals(t);  s.compareTo(t);
s.indexOf("ab");  s.isEmpty();
s.startsWith("ab");
s.endsWith("z");
s.contains("x"); // prefix /
suffix / substring tests
s.toLowerCase();
s.toUpperCase();
// case-normalize (e.g. case-
insensitive compare)
s.trim();  s.replace('a', 'b');
s.repeat(n);        // strip
ends / swap chars / repeat
StringBuilder builder = new
StringBuilder();
builder.append(x);
builder.reverse();
builder.toString();
builder.charAt(i);
builder.setCharAt(i, c);
builder.deleteCharAt(i);
builder.length();
String.join(",", list);
// list → "a,b,c"
```

```
Arrays.sort(arr);
// ascending in place
Arrays.sort(intervals, (a, b) -
> a[0] - b[0]);        // 2D by
first field
Arrays.fill(dp,
Integer.MAX_VALUE);
// seed a dp/dist array
Arrays.equals(window, need);
// ← element-wise array compare
(anagram check)
Arrays.copyOf(arr, len);
Arrays.copyOfRange(arr, i, j);
System.arraycopy(src, srcPos,
dst, dstPos, len);    // ←
fast block copy (merge step)
Arrays.asList(1, 2, 3);
// fixed-size List view
int sum =
Arrays.stream(arr).sum();
int max =
Arrays.stream(arr).max().getAsI
nt();
// List<Integer> ↔ int[]
int[] a =
list.stream().mapToInt(Integer:
:intValue).toArray();
List<Integer> l =
Arrays.stream(a).boxed().collec
t(Collectors.toList());
```

```
Math.max(a, b);  Math.min(a,
b);  Math.abs(x);
Math.pow(a, b);  Math.sqrt(x);
Math.ceil(x);  Math.floor(x);
n & (n - 1);            // clear
lowest set bit
n & (-n);               //
isolate lowest set bit
1 << i;                 // bit i
set  (use 1L << i for i ≥ 31)
(n >> i) & 1;           // read
bit i
(n & 1) == 0;           // ← even
(last bit 0)
(n & 1) == 1;           // ← odd
(last bit 1)
n >> 1;                 // ←
divide by 2 (drop last bit);  n
<< 1 = multiply by 2
Integer.bitCount(n);  // # of
set bits
Integer.toBinaryString(n);
```

```
tmap.firstKey();
tmap.lastKey();
tmap.floorKey(x);    // largest
key ≤ x
tmap.ceilingKey(x);  //
smallest key ≥ x
tmap.lowerKey(x);    // largest
key < x      tmap.higherKey(x);
// smallest key > x
tset.floor(x);
tset.ceiling(x);
tset.lower(x);  tset.higher(x);
tset.first();  tset.last();
```

```
// —— NEGATIVE-SAFE MODULO ——
replaces the manual ((x % n) +
n) % n
Math.floorMod(x, n);
// always in [0, n-1] even when
x < 0 (prefix-sum % k)
// —— int[] INSTEAD OF HashMap
for a bounded key space ——
int[] count = new int[26];
// O(1) no-boxing freq vs
HashMap<Character,Integer>
count[ch - 'a']++;
// also faster to compare:
Arrays.equals(a, b)
// —— 1D ROLLING ARRAY for DP
—— drop a dimension when row i
only needs row i-1
int[] dp = new int[n + 1];
// O(n) space instead of O(m·n)
for (...) for (int j = target;
j >= w; j--) dp[j] += dp[j -
w];  // 0/1 knapsack: iterate j
DOWN
// —— FIXED-SIZE HEAP for top-k
—— O(n log k) beats sorting O(n
log n)
if (heap.size() > k)
heap.poll();     // keep only k
best; min-heap for "k largest"
// —— LAZY DELETION over
pq.remove(obj) —— pq.remove is
O(n); skip stale entries
instead
if (entry[0] != dist[node])
continue; // Dijkstra: ignore
outdated heap entries
// —— ArrayDeque over Stack /
LinkedList —— faster, no
synchronization, both ends O(1)
Deque<Integer> stack = new
ArrayDeque<>();   // never use
java.util.Stack
// —— StringBuilder over String
+= in a loop —— O(n) vs O(n²)
StringBuilder builder = new
StringBuilder();      //
append, then builder.toString()
once
// —— EARLY EXIT / PRUNING ——
return the moment the answer is
decided
if (found) return result;
// e.g. backtracking bounds
```

Template Index

Two Pointers

Pattern 1 — Opposite Two Pointers

```
int i = 0, j = n - 1;
while (i < j) {
    if (condition == target) {
        /* record */ i++; j--;
    } else if (tooSmall) {
        i++;
    } else {
        j--;
    }
}
```

Sliding Window

The Three Templates

```
int i = 0;
for (int j = 0; j < n; j++) {
    // 1. add nums[j] to state
    if (j - i + 1 == k) {
        // 2. record (window is
exactly size k)
        // 3. remove nums[i]
from state
        i++;
    }
}
int i = 0, result = 0;
for (int j = 0; j < n; j++) {
    // 1. add nums[j] to state
    while (/* window exceeds
constraint */) {
        i++;
    }
    result = Math.max(result, j
- i + 1);
}
int i = 0, result =
Integer.MAX_VALUE;
for (int j = 0; j < n; j++) {
    // 1. add nums[j] to state
    while (/* window VALID */)
{
        result =
Math.min(result, j - i + 1);
        i++;
    }
}
```

Frequency Map Trick (used by 567 and 76)

```
if (need[s.charAt(j)]-- > 0)
required--;
if (need[s.charAt(i)]++ >= 0)
required++;
i++;
```

Prefix Sum + HashMap

Core Template

```
Map<Long, Integer> map = new
HashMap<>();
map.put(0L, ???);
long sum = 0;
for (int i = 0; i <
nums.length; i++) {
    sum += nums[i];
    long lookup =
transform(sum);
    map.put(storeKey(sum),
storeValue(i)); // what to
store (sum or sum%k → count or
index)
}
```

Binary Search

The Only Template — [i, j), find first k with condition(k)

```
int i = 0, j = nums.length;
while (i < j) {
    int k = i + (j - i) / 2;
    if (!condition(k)) {
        i = k + 1;
    } else {
        j = k;
    }
}
```

How to Choose

Sorting

Template 1 — Merge Sort

```
public void mergeSort(int[]
nums) {
    mergeSort(nums, 0,
nums.length, new
int[nums.length]);
}
private void mergeSort(int[]
nums, int start, int end, int[]
temp) {
    if (end - start <= 1)
return;
    int mid = start + (end -
start) / 2;
    mergeSort(nums, start, mid,
temp);
    mergeSort(nums, mid,   end,
temp);
    merge(nums, start, mid,
end, temp);
}
private void merge(int[] nums,
int start, int mid, int end,
int[] temp) {
    int i = start, j = mid, k =
start;
    while (i < mid || j < end)
{
        if (j >= end || (i <
mid && nums[i] <= nums[j])) {
            temp[k++] =
nums[i++];
        } else {
            temp[k++] =
nums[j++];
        }
    }
    for (i = start; i < end;
i++) nums[i] = temp[i];
}
```

Template 2 — Quick Sort (3-way Dutch Flag Partition)

```
public void quickSort(int[]
nums) {
    quickSort(nums, 0,
nums.length);
}
private void quickSort(int[]
nums, int start, int end) {
    if (end - start <= 1)
return;
    int pivot = nums[start +
(end - start) / 2];
    int i = start, k = start, j
= end - 1;
    while (k <= j) {
        if (nums[k] < pivot) {
            swap(nums, k++,
i++);
        } else if (nums[k] ==
pivot) {
            k++;
        } else {
            swap(nums, k, j--);
        }
    }
    quickSort(nums, start, i);
    quickSort(nums, k, end);
}
private void swap(int[] nums,
int i, int j) {
    int t = nums[i]; nums[i] =
nums[j]; nums[j] = t;
}
```

Template 3 — Quick Select (partial sort — k-th element)

```
private int quickSelect(int[]
nums, int start, int end, int
target) {
    int pivot = nums[start +
(end - start) / 2];
    int i = start, k = start, j
= end - 1;
    while (k <= j) {
        if (nums[k] < pivot) {
            swap(nums, k++,
i++);
        } else if (nums[k] ==
pivot) {
            k++;
        } else {
            swap(nums, k, j--);
        }
    }
    if (target < i) {
        return
quickSelect(nums, start, i,
target);
    } else if (target < k) {
        return pivot;
    } else {
        return
quickSelect(nums, k, end,
target);
    }
}
```

Template 4 — Counting Sort

```
public void countingSort(int[]
arr) {
    if (arr.length == 0)
return;
    int min =
Arrays.stream(arr).min().getAsI
nt();
    int max =
Arrays.stream(arr).max().getAsI
nt();
    int[] buckets = new int[max
- min + 1];
    for (int x : arr) buckets[x
- min]++;
    int i = 0;
    for (int v = 0; v <
buckets.length; v++)
        while (buckets[v]-- >
0) arr[i++] = v + min;
}
```

Linked List

When to Use — Signal → Pattern

All Four Templates

```
// 1. DELETE / FILTER  —
sentinel guards head removal;
previous tracks last kept node
ListNode sentinel = new
ListNode(0);
sentinel.next = head;
ListNode previous = sentinel,
current = head;
while (current != null) {
    if (shouldDelete(current))
{
        previous.next =
current.next;
    } else {
        previous = current;
    }
    current = current.next;
}
return sentinel.next;
// 2. REVERSAL — re-wire one
node at a time
ListNode previous = null,
current = head;
while (current != null) {
    ListNode next =
current.next;
    current.next = previous;
    previous = current;
    current = next;
}
return previous;
// 3. FAST / SLOW — two
pointers at different speeds
ListNode slow = head, fast =
head;
while (fast != null &&
fast.next != null) {
    slow = slow.next;
    fast = fast.next.next;
}
// 4. MERGE — sentinel collects
nodes from two lists in order
ListNode sentinel = new
ListNode(0);
ListNode current = sentinel;
while (a != null && b != null)
{
    if (a.val <= b.val) {
        current.next = a;
        a = a.next;
    } else {
        current.next = b;
        b = b.next;
    }
    current = current.next;
}
current.next = (a != null) ? a
: b;
return sentinel.next;
```

String

Core Idioms

```
// 1. CHAR COUNT — lowercase
letters
int[] count = new int[26];
for (char ch : s.toCharArray())
count[ch - 'a']++;
// 2. CHAR COUNT — digits 0–9
int[] count = new int[10];
for (char ch : s.toCharArray())
count[ch - '0']++;
// 3. CHAR TO INTEGER — build
number digit by digit
int num = 0;
for (int i = 0; i < s.length();
i++)
    num = num * 10 +
(s.charAt(i) - '0');
// 4. ANAGRAM HASH KEY —
frequency-based (bucket sort
style) O(k) vs sort O(k log k)
String hashKey(String s) {
    int[] count = new int[26];
    for (char ch :
s.toCharArray()) count[ch -
'a']++;
    StringBuilder builder = new
StringBuilder();
    for (int i = 0; i < 26;
i++) {
        builder.append((char)
('a' + i));

builder.append(count[i]);
    }
    return builder.toString();
}
char[] chars = s.toCharArray();
Arrays.sort(chars);
String key = new String(chars);
// 5. BUCKET SORT — sort by
frequency without comparison
sort
int[] count = new int[128];
for (char ch : s.toCharArray())
count[ch]++;
List<Character>[] buckets = new
List[s.length() + 1]; // index
= frequency
for (int i = 0; i < 128; i++) {
    if (count[i] > 0) {
        int freq = count[i];
        if (buckets[freq] ==
null) buckets[freq] = new
ArrayList<>();

buckets[freq].add((char) i);
    }
}
StringBuilder builder = new
StringBuilder();
for (int freq = buckets.length
- 1; freq > 0; freq--) {
    if (buckets[freq] == null)
continue;
    for (char ch :
buckets[freq])
        for (int i = 0; i <
freq; i++) builder.append(ch);
}
// 6. EXPAND FROM CENTER —
palindrome check in O(1) space
int expand(String s, int i, int
j) {
    while (i >= 0 && j <
s.length() && s.charAt(i) ==
s.charAt(j)) { i--; j++; }
    return j - i - 1;
}
```

Stack / Queue / Heap

All Templates

```
// 1. STACK (LIFO) — use
ArrayDeque, not Stack class
Deque<Integer> stack = new
ArrayDeque<>();
stack.push(x);
stack.pop();
stack.peek();
// 2A. MONOTONE DECREASING
STACK — next GREATER element
Deque<Integer> stack = new
ArrayDeque<>();
for (int i = 0; i < n; i++) {
    while (!stack.isEmpty() &&
nums[stack.peek()] < nums[i])
        result[stack.pop()] =
nums[i];
    stack.push(i);
}
// 2B. MONOTONE INCREASING
STACK — next SMALLER element /
span width
Deque<Integer> stack = new
ArrayDeque<>();
for (int i = 0; i < n; i++) {
    while (!stack.isEmpty() &&
nums[stack.peek()] > nums[i]) {
        int height =
nums[stack.pop()];
        int width  =
stack.isEmpty() ? i : i -
stack.peek() - 1;
    }
    stack.push(i);
}
// 3. MONOTONE DEQUE — sliding
window max/min
Deque<Integer> queue = new
ArrayDeque<>();
for (int i = 0; i < n; i++) {
    while (!queue.isEmpty() &&
nums[queue.peekLast()] <=
nums[i])
        queue.pollLast();
    queue.offerLast(i);
    if (queue.peekFirst() < i -
k + 1) queue.pollFirst();
    if (i >= k - 1) result[i -
k + 1] =
nums[queue.peekFirst()];
}
// 4. PRIORITY QUEUE
PriorityQueue<Integer> minPQ =
new PriorityQueue<>();
PriorityQueue<Integer> maxPQ =
new PriorityQueue<>
(Collections.reverseOrder());
// max-heap
PriorityQueue<int[]>    pq   =
new PriorityQueue<>((a, b) ->
a[1] - b[1]); // custom: sort
by index 1
pq.offer(x);
pq.poll();
pq.peek();
// 5. TWO HEAPS — maintain
median in O(log n)
PriorityQueue<Integer> maxHeap
= new PriorityQueue<>
(Collections.reverseOrder());
PriorityQueue<Integer> minHeap
= new PriorityQueue<>();
void addNum(int num) {
    maxHeap.offer(num);

minHeap.offer(maxHeap.poll());
    if (maxHeap.size() <
minHeap.size())

maxHeap.offer(minHeap.poll());
}
double findMedian() {
    return maxHeap.size() >
minHeap.size()
        ? maxHeap.peek()
        : (maxHeap.peek() +
minHeap.peek()) / 2.0;
}
```

BFS (Tree)

The Template — Level-Aware BFS

```
Queue<TreeNode> queue = new
ArrayDeque<>();
queue.offer(root);
int level = 0;
while (!queue.isEmpty()) {
    int size = queue.size();
    level++;
    for (int i = 0; i < size;
i++) {
        TreeNode node =
queue.poll();
        if (node.left  != null)
queue.offer(node.left);
        if (node.right != null)
queue.offer(node.right);
    }
}
```

When to Use — Signal → Pattern

The One Rule — Snapshot Level Size First

```
Always snapshot the level size
BEFORE the inner for loop:
    int size = queue.size();
    for (int i = 0; i < size;
i++) { ... }
Without the snapshot, newly
added children mix with the
current level
and i == size-1 / i == 0 checks
become meaningless.
```

DFS / Backtracking

When to Use — Signal → Pattern

All Templates

```
// 1. PROCESS CHILDREN FIRST,
COMBINE AT NODE (post-order)
private int dfs(TreeNode node)
{
    if (node == null) return
BASE;
    int left  = dfs(node.left);
    int right =
dfs(node.right);
    return ...;
}
// 2. PROCESS NODE FIRST, PASS
STATE DOWN (pre-order)
private void dfs(TreeNode node,
int accumulated) {
    if (node == null) return;
    accumulated += node.val;
    if (isLeaf(node)) { /*
record */ return; }
    dfs(node.left,
accumulated);
    dfs(node.right,
accumulated);
}
// 3. TREE — GLOBAL ANSWER
(return up the best single arm;
update global across both arms)
int answer = Integer.MIN_VALUE;
private int dfs(TreeNode node)
{
    if (node == null) return 0;
    int left  = Math.max(0,
dfs(node.left));
    int right = Math.max(0,
dfs(node.right));
    answer = Math.max(answer,
left + right + node.val);
    return Math.max(left,
right) + node.val;
}
// 4. GRID DFS — mark visited
by mutating grid; 4-directional
flood fill
private void dfs(int[][] grid,
int r, int c) {
    if (r < 0 || r >=
grid.length || c < 0 || c >=
grid[0].length) return;
    if (grid[r][c] != TARGET)
return;
    grid[r][c] = VISITED;
    dfs(grid, r+1, c);
dfs(grid, r-1, c);
    dfs(grid, r, c+1);
dfs(grid, r, c-1);
}
// 5. GRAPH DFS — 3-color
visited: 0=unseen 1=in-stack
2=done
int[] state;
private boolean dfs(int node) {
    if (state[node] == 1)
return true;
    if (state[node] == 2)
return false;
    state[node] = 1;
    for (int neighbor :
graph.get(node))
        if (dfs(neighbor))
return true;
    state[node] = 2;
    return false;
}
// 6. BACKTRACKING — choose /
recurse / undo
private void backtrack(int
start, List<Integer> current) {
    if (baseCase) {
result.add(new ArrayList<>
(current)); return; }
    for (int i = start; i < n;
i++) {
        if (skipCondition(i))
continue;
        current.add(nums[i]);
        backtrack(next,
current);

current.remove(current.size()-1
);
    }
}
```

Graph

Complexity Legend

```
BFS / DFS         O(V + E)
visit each vertex and edge once
Topo sort (Kahn)  O(V + E)
each node enqueued once, each
edge relaxed once
Union-Find        ~O(n·α(n)) ≈
O(n)   α = inverse Ackermann,
effectively constant
Dijkstra          O((V + E) log
V)    non-negative weights ONLY
```

Graph Representation

```
List<List<Integer>> graph = new
ArrayList<>();
for (int i = 0; i < n; i++)
graph.add(new ArrayList<>());
for (int[] edge : edges) {

graph.get(edge[0]).add(edge[1])
;

graph.get(edge[1]).add(edge[0])
;
}
List<List<int[]>> graph = new
ArrayList<>();
for (int i = 0; i < n; i++)
graph.add(new ArrayList<>());
for (int[] edge : edges)
    graph.get(edge[0]).add(new
int[]{edge[1], edge[2]});
int[] dr = {1, -1, 0,  0};
int[] dc = {0,  0, 1, -1};
```

Core Templates

```
// 1. BFS — shortest path /
level order (track distance by
level-size snapshot)
Queue<Integer> queue = new
ArrayDeque<>();
boolean[] visited = new
boolean[n];
queue.offer(start);
visited[start] = true;
int level = 0;
while (!queue.isEmpty()) {
    int size = queue.size();
    for (int i = 0; i < size;
i++) {
        int node =
queue.poll();
        for (int neighbor :
graph.get(node)) {
            if
(!visited[neighbor]) {

visited[neighbor] = true;

queue.offer(neighbor);
            }
        }
    }
    level++;
}
// 2. TOPOLOGICAL SORT — Kahn's
BFS
int[] inDegree = new int[n];
for (int[] edge : edges)
inDegree[edge[1]]++;
Queue<Integer> queue = new
ArrayDeque<>();
for (int i = 0; i < n; i++) if
(inDegree[i] == 0)
queue.offer(i);
List<Integer> order = new
ArrayList<>();
while (!queue.isEmpty()) {
    int node = queue.poll();
    order.add(node);
    for (int neighbor :
graph.get(node))
        if (--
inDegree[neighbor] == 0)
queue.offer(neighbor);
}
// 3. UNION FIND
int[] parent, rank;
void init(int n) {
    parent = new int[n]; rank =
new int[n];
    for (int i = 0; i < n; i++)
parent[i] = i;
}
int find(int x) {
    if (parent[x] != x)
parent[x] = find(parent[x]);
    return parent[x];
}
boolean union(int x, int y) {
    int px = find(x), py =
find(y);
    if (px == py) return false;
    if (rank[px] < rank[py]) {
int t = px; px = py; py = t; }
    parent[py] = px;
    if (rank[px] == rank[py])
rank[px]++;
    return true;
}
// 4. DIJKSTRA — shortest path
(non-negative weights)
int[] dist = new int[n];
Arrays.fill(dist,
Integer.MAX_VALUE);
dist[src] = 0;
PriorityQueue<int[]> pq = new
PriorityQueue<>((a, b) -> a[1]
- b[1]);
pq.offer(new int[]{src, 0});
while (!pq.isEmpty()) {
    int[] current = pq.poll();
    int node = current[0], d =
current[1];
    if (d > dist[node])
continue;
    for (int[] nb :
graph.get(node)) {
        int next = nb[0], w =
nb[1];
        if (dist[node] + w <
dist[next]) {
            dist[next] =
dist[node] + w;
            pq.offer(new int[]
{next, dist[next]});
        }
    }
}
// 5. BIPARTITE CHECK — BFS 2-
coloring
int[] color = new int[n];
Arrays.fill(color, -1);
Queue<Integer> queue = new
ArrayDeque<>();
for (int start = 0; start < n;
start++) {
    if (color[start] != -1)
continue;
    color[start] = 0;
    queue.offer(start);
    while (!queue.isEmpty()) {
        int node =
queue.poll();
        for (int neighbor :
graph.get(node)) {
            if (color[neighbor]
== -1) { color[neighbor] = 1 -
color[node];
queue.offer(neighbor); }
            else if
(color[neighbor] ==
color[node]) return false;
        }
    }
}
return true;
```

Greedy

Two Interval Templates

```
MERGE DIRECTION RULE (for
merge-style problems like #56,
where both keys work):
    Sort by START  →  traverse
FORWARD  (i = 0 → n-1), extend
the END of last kept
    Sort by END    →  traverse
BACKWARD (i = n-1 → 0), extend
the START of last kept
    These two are exact mirror
images. See #56 for both
written out side by side.
    (NOTE: this mirror only
applies to merging. Template 1
below sorts by end but
     still sweeps FORWARD -
there the goal is counting, not
merging.)
TEMPLATE 1 - Sort by END time,
sweep forward, track end of
last kept interval
                   Use when:
maximizing count of non-
overlapping intervals
                   Greedy insight:
earliest-finishing interval
leaves most room for future
ones
TEMPLATE 2 - Sort by START
time, sweep forward, merge when
overlapping
                   Use when:
merging/counting overlapping
intervals
                   Greedy insight:
processing in start order →
each interval only needs to
                   compare with the
last merged result
```

```
Arrays.sort(intervals, (a, b) -
> a[1] - b[1]);
int lastEnd =
Integer.MIN_VALUE;
for (int[] interval :
intervals) {
    if (interval[0] >= lastEnd)
{
        lastEnd = interval[1];
    } else {
    }
}
Arrays.sort(intervals, (a, b) -
> a[0] - b[0]);
List<int[]> result = new
ArrayList<>();
for (int[] interval :
intervals) {
    if (result.isEmpty() ||
result.get(result.size()-1)[1]
< interval[0]) {
        result.add(interval);
    } else {

result.get(result.size()-1)[1]
=

Math.max(result.get(result.size
()-1)[1], interval[1]);
    }
}
Arrays.sort(intervals, (a, b) -
> a[0] - b[0]);
PriorityQueue<Integer> pq = new
PriorityQueue<>();
for (int[] interval :
intervals) {
    if (!pq.isEmpty() &&
pq.peek() <= interval[0])
        pq.poll();
    pq.offer(interval[1]);
}
```

Dynamic Programming

All Six Templates

```
// 1. LINEAR 1D — each cell
depends on a fixed number of
previous cells
int[] dp = new int[n + 1];
dp[0] = base;
for (int i = 1; i <= n; i++)
    dp[i] = f(dp[i-1], dp[i-2],
...);
// 2A. LOOK-BACK 1D — dp[i]
depends on all j < i
int[] dp = new int[n];
for (int i = 0; i < n; i++) {
    for (int j = 0; j < i; j++)
{
        dp[i] = f(dp[i],
dp[j]);
    }
}
// 3. 2D SEQUENCE — match
consumes both, mismatch drops
one side. WHEN: LCS / edit
// 4. INTERVAL DP — short
ranges first (i right-to-left,
j from i+1; dp[i+1][*] ready).
// 5. GRID DP — each cell
accumulates from cells it could
arrive from (up/left).
// 6. STATE MACHINE — named
states updated from yesterday's
states each step.
```

```
Want count or min/max?
├── Count (dp[j] += dp[j -
num])
│   ├── 0/1 (each item once):
j descending → #416, #494
│   └── Unbounded (reuse):
j ascending  → #518
│       └── Ordered (permuts):
target outer, nums inner → #377
└── Min/Max (dp[j] =
min/max(...))
    ├── 0/1 minimize:
j descending → #474 (maximize)
    └── Unbounded minimize:
j ascending  → #322
```

```
// —— 0/1 KNAPSACK —— each item
at most once
int[] dp = new int[target + 1];
dp[0] = 1;
for (int i = 0; i <
nums.length; i++) {
    for (int j = target; j >=
nums[i]; j--)
        dp[j] += dp[j -
nums[i]];
}
// —— UNBOUNDED KNAPSACK ——
each item reusable
int[] dp = new int[target + 1];
dp[0] = 1;
for (int i = 0; i <
nums.length; i++) {
    for (int j = nums[i]; j <=
target; j++)
        dp[j] += dp[j -
nums[i]];
}
// —— PERMUTATION COUNT ——
order matters (Combination Sum
IV)
int[] dp = new int[target + 1];
dp[0] = 1;
for (int j = 1; j <= target;
j++) {
    for (int num : nums) {
        if (j >= num) dp[j] +=
dp[j - num];
    }
}
```

```
int[][] dp = new int[m + 1][n +
1];
for (int i = 1; i <= m; i++) {
    for (int j = 1; j <= n;
j++) {
        if (s1.charAt(i-1) ==
s2.charAt(j-1)) {
            dp[i][j] = dp[i-1]
[j-1] + 1;
        } else {
            dp[i][j] =
combine(dp[i-1][j], dp[i][j-1],
dp[i-1][j-1]);
        }
    }
}
```

```
int[][] dp = new int[n][n];
for (int i = 0; i < n; i++)
dp[i][i] = base_case;
for (int i = n - 1; i >= 0; i-
-) {
    for (int j = i + 1; j < n;
j++) {
        dp[i][j] = ...;
    }
}
```

```
int[][] dp = new int[n][n];
for (int i = 0; i < n; i++)
dp[i][i] = base_case;
for (int i = 0; i < n; i++) {
    for (int j = i - 1; j >= 0;
j--) {
        dp[i][j] = ...;
    }
}
```

```
int[][] dp = new int[n][n];
for (int i = 0; i < n; i++)
dp[i][0] = 1;
for (int i = 1; i < n; i++) {
    for (int j = 1; j <= i;
j++) {
        dp[i][j] = ...;
    }
}
```

```
int[] dr = {1,-1,0,0}, dc =
{0,0,1,-1};
int[][] dp = new int[rows]
[cols];
for (int i = 0; i < rows; i++)
    for (int j = 0; j < cols;
j++)
        dp[i][j] = grid[i][j] +
f(dp[i-1][j], dp[i][j-1]);
int[][] memo = new int[rows]
[cols];
for (int i = 0; i < rows; i++)
    for (int j = 0; j < cols;
j++)
        dfs(grid, i, j, memo,
dr, dc);
```

```
cash      = max profit when NOT
holding (free to buy or idle)
hold      = max profit when
HOLDING stock (bought it at
some cost)
cooldown  = max profit right
after selling (can't buy today)
```

```
                          buy
sell                      re-buy
condition
#121 (1 tx):          hold =
max(hold, -price)
no re-buy (ignore cash)
#122 (unlimited):   hold =
max(hold, cash - price)
re-buy with full cash
#123 (2 tx):          chain: buy2
uses sell1 cash               4
states chained
#309 (cooldown):    hold =
max(hold, cooldown - price)
must wait 1 day
#714 (fee):           hold =
max(hold, cash - price)
pay fee on sell
```

Bit Manipulation

Canonical Template

```
// —— XOR CANCEL —— pairs
cancel; lone element survives
int result = 0;
for (int num : nums) result ^=
num;
// —— BIT OPERATIONS ——
boolean isSet  = ((n >> i) & 1)
== 1;
int set        = n | (1 << i);
int clear      = n & ~(1 << i);
int toggle     = n ^ (1 << i);
int lowest     = n & (-n);
boolean isPow2 = n > 0 && (n &
(n-1)) == 0;
// —— COUNT SET BITS —— Brian
Kernighan: each iteration
removes one set bit
int count = 0;
while (n != 0) { count++; n &=
(n - 1); }
// —— BITMASK AS SET —— 26-bit
integer represents presence of
26 letters
int mask = 0;
for (char c :
word.toCharArray()) mask |= (1
<< (c - 'a'));
if ((mask[i] & mask[j]) == 0) {
/* no shared letter */ }
```

Variations

```
int ones = 0, twos = 0;
for (int num : nums) {
    ones = (ones ^ num) &
~twos;
    twos = (twos ^ num) &
~ones;
}
int xor  = 0;
for (int num : nums) xor ^=
num;
int diff = xor & (-xor);
int a = 0;
for (int num : nums)
    if ((num & diff) != 0) a ^=
num;
int b = xor ^ a;
int[] dp = new int[n + 1];
for (int i = 1; i <= n; i++)
    dp[i] = dp[i >> 1] + (i &
1);
int shift = 0;
while (left != right) { left
>>= 1; right >>= 1; shift++; }
return left << shift;
int add(int a, int b) {
    while (b != 0) {
        int carry = a & b;
        a = a ^ b;
        b = carry << 1;
    }
    return a;
}
```

Design

Canonical Template — HashMap + Doubly Linked List (LRU)

```
class CacheNode {
    int key, value;
    CacheNode previous, next;
    CacheNode(int key, int
value) { this.key = key;
this.value = value; }
}
// —— SENTINEL NODES —— head
(MRU side) ↔ ... ↔ tail (LRU
side)
CacheNode head = new
CacheNode(0, 0), tail = new
CacheNode(0, 0);
head.next = tail; tail.previous
= head;
// —— REMOVE NODE ——
void remove(CacheNode node) {
    node.previous.next =
node.next;
    node.next.previous =
node.previous;
}
// —— INSERT AFTER HEAD (= mark
as MRU) ——
void insertAfterHead(CacheNode
node) {
    node.next = head.next;
    node.previous = head;
    head.next.previous = node;
    head.next = node;
}
// —— LRU EVICTION ——
tail.previous is LRU
CacheNode lru = tail.previous;
remove(lru);
map.remove(lru.key);
```

Variations

Math

Canonical Templates

```
// —— DIGIT EXTRACTION LOOP ——
peel digits right-to-left
while (n != 0) {
    int digit = n % 10;
    n /= 10;
}
// —— FAST POWER
(exponentiation by squaring) ——
double fastPow(double x, long
n) {
    double result = 1;
    while (n > 0) {
        if ((n & 1) == 1)
result *= x;
            x *= x;
            n >>= 1;
    }
    return result;
}
// —— BASE CONVERSION ——
generic base b, 0-indexed
StringBuilder builder = new
StringBuilder();
while (num > 0) {
    builder.append(num % b);
    num /= b;
}
builder.reverse();
int value = 0;
for (int digit : digits) value
= value * b + digit;
// —— SIEVE OF ERATOSTHENES ——
mark composites up to n
boolean[] composite = new
boolean[n];
for (int i = 2; i * i < n; i++)
{
    if (composite[i]) continue;
    for (int j = i * i; j < n;
j += i)
        composite[j] = true;
}
```